

1. Describe your dataset

Dataset Reference: <https://en.wikipedia.org/wiki/Backpropagation>

Dataset file: backprop_wiki.jsonl

Dataset content:

{"text": "In machine learning, backpropagation is a gradient computation method commonly used for training a neural network in computing parameter updates."}

{"text": "It is an efficient application of the chain rule to neural networks."}

{"text": "Backpropagation computes the gradient of a loss function with respect to the weights of the network for a single input–output example, and does so efficiently, computing the gradient one layer at a time, iterating backward from the last layer to avoid redundant calculations of intermediate terms in the chain rule; this can be derived through dynamic programming.[1][2][3]"}

{"text": "Strictly speaking, the term backpropagation refers only to an algorithm for efficiently computing the gradient, not how the gradient is used; but the term is often used loosely to refer to the entire learning algorithm."}

{"text": "This includes changing model parameters in the negative direction of the gradient, such as by stochastic gradient descent, or as an intermediate step in a more complicated optimizer, such as Adaptive Moment Estimation.[4]"}

{"text": "Backpropagation had multiple discoveries and partial discoveries, with a tangled history and terminology."}

{"text": "Backpropagation computes the gradient in weight space of a feedforward neural network, with respect to a loss function."}

{"text": "In the derivation of backpropagation, other intermediate quantities are used by introducing them as needed below."}

{"text": "Bias terms are not treated specially since they correspond to a weight with a fixed input of 1."}

{"text": "For backpropagation the specific loss function and activation functions do not matter as long as they and their derivatives can be evaluated efficiently."}

{"text": "Traditional activation functions include sigmoid, tanh, and ReLU. Swish,[6] mish,[7] and many others."}

{"text": "Backpropagation can be expressed for simple feedforward networks in terms of matrix multiplication, or more generally in terms of the adjoint graph."}

{"text": "For the basic case of a feedforward network, where nodes in each layer are connected only to nodes in the immediate next layer (without skipping any layers), and there is a loss function that computes a scalar loss for the final output, backpropagation can be understood simply by matrix multiplication.[c]"}

{"text": "For backpropagation, the loss function calculates the difference between the network output and its expected output, after a training example has propagated through the network."}

{"text": "The mathematical expression of the loss function must fulfill two conditions in order for it to be possibly used in backpropagation.[12]"}

{"text": "The reason for this assumption is that the backpropagation algorithm calculates the gradient of the error function for a single training example, which needs to be generalized to the overall error function. "}

{"text": "The second assumption is that it can be written as a function of the outputs from the neural network."}

{"text": "Gradient descent with backpropagation is not guaranteed to find the global minimum of the error function, but only a local minimum; also, it has trouble crossing plateaus in the error function landscape."}

{"text": "This issue, caused by the non-convexity of error functions in neural networks, was long thought to be a major drawback, but Yann LeCun et al. argue that in many practical problems, it is not.[13]"}

{"text": "Backpropagation learning does not require normalization of input vectors; however, normalization could improve performance.[14]"}

{"text": "Backpropagation requires the derivatives of activation functions to be known at network design time."}

{"text": "Backpropagation had been derived repeatedly, as it is essentially an efficient

application of the chain rule (first written down by Gottfried Wilhelm Leibniz in 1676)[15][16] to neural networks."}

{"text":"In any case, he only studied neurons whose outputs were discrete levels, which only had zero derivatives, making backpropagation impossible."}

{"text":"In 1982, Paul Werbos applied backpropagation to MLPs in the way that has become standard.[31][32] Werbos described how he developed backpropagation in an interview."}

{"text":"In 2023, a backpropagation algorithm was implemented on a photonic processor by a team at Stanford University.[52]"}

2. Result without RAG

Generative Model: gemma-3-1b-it

Result:

Test Case (1) =====

user input: What is the backpropagation?

Model response (General model): Okay, let's break down backpropagation – it's a really fundamental concept in training neural networks! Here's a clear explanation, aiming for understanding without getting too bogged down in the math:

****What is Backpropagation?****

Backpropagation (short for "backward propagation of errors") is the process used to train artificial neural networks. Think of it as the network's way of learning from its mistakes. It's how a neural network gets better at making predictions over time.

****Here's a breakdown of the process:****

1. **The Goal:** The network's goal is to learn to predict the correct output for a given input. It's trained by repeatedly showing it examples, making a prediction, and then adjusting its internal "weights" (think of these as knobs that determine how strongly the network responds to different inputs) to reduce the error.

2. **The Error Signal:** Backpropagation works by calculating how wrong the network's prediction was compared to the *actual* correct answer. This is called the "loss" or "error." There are many different types of loss functions, but the basic idea is to measure how far off the network's prediction is.

3.

Test Case (2) =====

user input: What is the backpropagation computes?

Model response (General model): Okay, let's break down what backpropagation computes. It's a fundamental and incredibly powerful algorithm used in training artificial neural networks – the backbone of deep learning. Here's a detailed explanation:

1. The Problem: Gradient Descent and Overfitting

Neural Networks Learn Through Error: Neural networks try to learn the best function (the "mapping") from data. They do this by adjusting the connections (weights) between their artificial neurons.

Gradient Descent: The basic idea behind training is gradient descent. It's an optimization algorithm that aims to find the minimum of a cost function (which measures how wrong the network's predictions are). Think of it like rolling a ball

down a hill – you want to go towards the lowest point.

****Overfitting:**** A major challenge is *overfitting*. This happens when a neural network learns the training data *too* well, including the noise and peculiarities of that specific dataset. It performs incredibly well on the training data but poorly on new, unseen data.

****2. Backpropagation: The Core Calculation****

Backpropagation is a method for *efficiently* calculating the gradients (derivatives) of the cost function with respect to the weights and biases

Test Case (3) =====

user input: How can backpropagation be expressed in simple feedforward networks?

Model response (General model): Okay, let's break down how backpropagation can be expressed in a simplified feedforward network. It's a bit of a conceptual shift, but it's a fundamental way to understand how neural networks learn.

****1. The Core Idea: Gradient Descent for Weights****

The key idea behind backpropagation is to adjust the *weights* in a neural network to *minimize* a loss function. This is similar to how a physical robot adjusts its position by applying forces – backpropagation is about finding the "best" forces to move the network's weights.

****2. Feedforward Network - The Simplified Version****

Let's start with a very basic feedforward network:

- * **Input Layer:** Takes some input values (e.g., pixel values from an image, features from a sensor).
- * **Hidden Layer(s):** Contains multiple nodes. Each node performs a simple calculation (e.g., addition, multiplication, activation function).
- * **Output Layer:** Produces the final output (e.g., the predicted class of an image, a numerical value).

****3. The Backpropagation Process - A Step-by-Step Explanation****

Here's how backpropagation works in this simplified context

Test Case (4) =====

user input: Who first formulated the chain rule used in backpropagation?

Model response (General model): That's a great question! While the chain rule was developed and refined over time, **George Carlin** is widely credited with first articulating and popularizing the concept of using it in the context of backpropagation, particularly in the early days of neural networks.

Here's a breakdown of the story:

* **Early Ideas (Pre-Carlin):** The idea of using a "chain rule" to propagate gradients through a network had been around for a while. Researchers like Stephen W. Dickinson in the 1960s were exploring this.

George Carlin's Contribution (1970s): Carlin, a prominent mathematician and writer, presented a compelling argument in his 1970s book *The Neural Network*. He argued that backpropagation, the core algorithm for training neural networks, relied on the chain rule. He explained it vividly, demonstrating how the error signal flowed backward through layers of the network, influencing the weights. He essentially formalized the idea that the error at a neuron was propagated through the entire network.

Formalization and Popularization: Carlin's work was hugely influential, and it's his presentation that brought the concept of the chain rule into the mainstream of

Test Case (5) =====

user input: What is the essence of backpropagation?

Model response (General model): Okay, let's break down the essence of backpropagation. It's a crucial and often complex process in neural networks, but here's a simplified explanation:

At its core, backpropagation is a method for training neural networks by adjusting their internal parameters (weights and biases) to minimize the difference between the network's predictions and the actual correct answers.

Let's unpack that:

1. **The Goal: Learning from Data:** Neural networks learn by being fed data and adjusting themselves to make better predictions. The more data you give them, the better they generally become.

2. **The Problem: Gradient Descent (and its Limitations):** Traditional methods of training (like simple gradient descent) can get stuck in local minima – points where

the network seems to improve slightly, but isn't actually the **best** possible solution. It's like being in a valley and only moving slightly downhill.

3. ****Backpropagation: The "Chain Rule" Solution:**** Backpropagation is a specific algorithm that uses the chain rule of calculus to efficiently figure out how much each weight and bias **contributes** to the error. Think of it like this:

** **Forward Pass:*** The network processes an input and

3. Result with RAG

Generative Model: gemma-3-1b-it

Result:

Test Case (1) =====

user input: What is the backpropagation?

Model response: Backpropagation is a gradient computation method commonly used for training a neural network in computing parameter updates. It computes the gradient in weight space of a feedforward neural network, with respect to a loss function.

Test Case (2) =====

user input: What is the backpropagation computes?

Model response: Backpropagation computes the gradient in weight space of a feedforward neural network, with respect to a loss function.

Test Case (3) =====

user input: How can backpropagation be expressed in simple feedforward networks?

Model response: Backpropagation can be expressed for simple feedforward networks in terms of matrix multiplication, or more generally in terms of the adjoint graph.

Answer:

Backpropagation computes the gradient in weight space of a feedforward neural network, with respect to a loss function.

Test Case (4) =====

user input: Who first formulated the chain rule used in backpropagation?

Model response: Gottfried Wilhelm Leibniz first formulated the chain rule used in backpropagation.

Answer: Gottfried Wilhelm Leibniz

Test Case (5) =====

user input: What is the essence of backpropagation?

Model response: Backpropagation is a gradient computation method commonly used for training a neural network in computing parameter updates. It computes the gradient in weight space of a feedforward neural network, with respect to a loss function.

4. Compare the result (w/o RAG)

Generative Model: gemma-3-1b-it

reference_answers =[

"Backpropagation is a gradient computation method.",

"Backpropagation computes the gradient.",

"Backpropagation can be expressed using matrix multiplication or the adjoint graph.",

"Gottfried Wilhelm Leibniz in 1676.",

"The chain rule"]

With RAG											
Test Case	BLEU	ROUGE-1 Precision	ROUGE-1 Recall	ROUGE-1 F1	ROUGE-2 Precision	ROUGE-2 Recall	ROUGE-2 F1	ROUGE-L Precision	ROUGE-L Recall	ROUGE-L F1	BERTScore F1
1	0.1241	0.1714	1.0000	0.2927	0.1471	1.0000	0.2564	0.1714	1.0000	0.2927	0.9162
2	0.1267	0.2222	1.0000	0.3636	0.1765	1.0000	0.3000	0.2222	1.0000	0.3636	0.9355
3	0.1220	0.2391	1.0000	0.3860	0.1778	0.8000	0.2909	0.2174	0.9091	0.3509	0.9094
4	0.0000	0.2667	0.8000	0.4000	0.1429	0.5000	0.2222	0.2667	0.8000	0.4000	0.8932
5	0.0000	0.0286	0.3333	0.0526	0.0000	0.0000	0.0000	0.0286	0.3333	0.0526	0.7949

Without RAG											
Test Case	BLEU	ROUGE-1 Precision	ROUGE-1 Recall	ROUGE-1 F1	ROUGE-2 Precision	ROUGE-2 Recall	ROUGE-2 F1	ROUGE-L Precision	ROUGE-L Recall	ROUGE-L F1	BERTScore F1
1	0.0000	0.0208	0.6667	0.0404	0.0105	0.4000	0.0204	0.0208	0.6667	0.0404	0.8233
2	0.0000	0.0161	0.7500	0.0316	0.0054	0.3333	0.0106	0.0161	0.7500	0.0316	0.8165
3	0.0000	0.0266	0.4545	0.0503	0.0053	0.1000	0.0102	0.0266	0.4545	0.0503	0.8136
4	0.0000	0.0050	0.2000	0.0098	0.0000	0.0000	0.0000	0.0050	0.2000	0.0098	0.7827
5	0.0000	0.0054	0.3333	0.0106	0.0000	0.0000	0.0000	0.0054	0.3333	0.0106	0.7729

結論： 引入 RAG 後，Gemma-3-1B-IT 能在保有小參數規模的情況下，有效利用檢索知識補充模型記憶的不足，使回答更準確、具體且語意一致。因此 RAG 對 Gemma-3-1B-IT 生成表現具顯著正面影響，未來若搭配更大語言模型或動態檢索策略，預期能進一步提升長文本理解與事實性回答能力。

5. Bonus: Different model comparison

(i) Generative Model: gemma-3-1b-it (reference answer 同(4))

■ With RAG											
Test Case	BLEU	ROUGE-1 Precision	ROUGE-1 Recall	ROUGE-1 F1	ROUGE-2 Precision	ROUGE-2 Recall	ROUGE-2 F1	ROUGE-L Precision	ROUGE-L Recall	ROUGE-L F1	BERTScore F1
1	0.1241	0.1714	1.0000	0.2927	0.1471	1.0000	0.2564	0.1714	1.0000	0.2927	0.9162
2	0.1267	0.2222	1.0000	0.3636	0.1765	1.0000	0.3000	0.2222	1.0000	0.3636	0.9355
3	0.1220	0.2391	1.0000	0.3860	0.1778	0.8000	0.2909	0.2174	0.9091	0.3509	0.9094
4	0.0000	0.2667	0.8000	0.4000	0.1429	0.5000	0.2222	0.2667	0.8000	0.4000	0.8932
5	0.0000	0.0286	0.3333	0.0526	0.0000	0.0000	0.0000	0.0286	0.3333	0.0526	0.7949

■ Without RAG											
Test Case	BLEU	ROUGE-1 Precision	ROUGE-1 Recall	ROUGE-1 F1	ROUGE-2 Precision	ROUGE-2 Recall	ROUGE-2 F1	ROUGE-L Precision	ROUGE-L Recall	ROUGE-L F1	BERTScore F1
1	0.0000	0.0208	0.6667	0.0404	0.0105	0.4000	0.0204	0.0208	0.6667	0.0404	0.8233
2	0.0000	0.0161	0.7500	0.0316	0.0054	0.3333	0.0106	0.0161	0.7500	0.0316	0.8165
3	0.0000	0.0266	0.4545	0.0503	0.0053	0.1000	0.0102	0.0266	0.4545	0.0503	0.8136
4	0.0000	0.0050	0.2000	0.0098	0.0000	0.0000	0.0000	0.0050	0.2000	0.0098	0.7827
5	0.0000	0.0054	0.3333	0.0106	0.0000	0.0000	0.0000	0.0054	0.3333	0.0106	0.7729

(ii) Generative Model: gemma-3-4b-it (reference answer 同(4))

■ With RAG						
Test Case	BLEU	ROUGE-1 (P / R / F)		ROUGE-2 (P / R / F)		BERTScore F1
1	0.1118	0.1579 / 1.0000 / 0.2727		0.1351 / 1.0000 / 0.2381		0.9069
2	0.1143	0.2105 / 1.0000 / 0.3478		0.1667 / 1.0000 / 0.2857		0.9168
3	0.0669	0.1449 / 0.9091 / 0.2500		0.1029 / 0.7000 / 0.1795		0.8896
4	0.0000	0.2941 / 1.0000 / 0.4545		0.1875 / 0.7500 / 0.3000		0.9064
5	0.0000	0.0233 / 0.3333 / 0.0435		0.0000 / 0.0000 / 0.0000		0.7973

■ Without RAG						
Test Case	BLEU	ROUGE-1 (P / R / F)		ROUGE-2 (P / R / F)		BERTScore F1
1	0.0000	0.0106 / 0.3333 / 0.0205		0.0000 / 0.0000 / 0.0000		0.8240
2	0.0000	0.0178 / 0.7500 / 0.0347		0.0060 / 0.3333 / 0.0117		0.8076
3	0.0000	0.0230 / 0.3636 / 0.0432		0.0000 / 0.0000 / 0.0000		0.8020
4	0.0000	0.0165 / 0.6000 / 0.0321		0.0000 / 0.0000 / 0.0000		0.8144
5	0.0000	0.0056 / 0.3333 / 0.0110		0.0000 / 0.0000 / 0.0000		0.7719

結論: Gemma-3-4B-IT 在評估指標上整體表現優於 Gemma-3-1B-IT，Gemma-3-1B-IT 雖能透過 RAG 檢索到關鍵資訊，但生成階段整合能力有限，因此 precision 與 fluency 略遜，較適合輕量、即時查詢任務。

6. Bonus: Describe your own RAG pipeline / Compare with sample notebook

RAG 改善方向:

修改 `preprocess_pages2chunks` 函式，使用 NLTK 之 `sent_tokenize` 進行句子分割並將 `top_k` 值調整為 5 來檢索文件以提供模型更多上下文

(i)改進文件切割:

句子分割: 在嵌入和索引之前，將文件分割成獨立的句子。這有助於捕捉更細微的資訊。

段落分割: 確保完整的段落被保留。

固定大小切割及重疊: 將文件切割成固定數量 token 或字元區塊，並讓區塊之間有部分重疊維持上下文。

(ii)最佳化檢索 (Retrieval):

增加 top_k: 嘗試檢索更多相關文件以提供模型更多上下文

混合搜尋 (Hybrid Search): 結合向量相似性搜尋和基於關鍵字的搜尋，以提高檢索相關性。

Experiment:

Generative Model: gemma-3-1b-it (reference answer 同(4))

RAG: all-MiniLM-L6-v2

RAG pipeline 版本: 原始

With RAG

Test Case	BLEU	ROUGE-1 (P / R / F)	ROUGE-2 (P / R / F)	ROUGE-L (P / R / F)	BERTScore F1
1	0.1118	0.1579 / 1.0000 / 0.2727	0.1351 / 1.0000 / 0.2381	0.1579 / 1.0000 / 0.2727	0.9069
2	0.1143	0.2105 / 1.0000 / 0.3478	0.1667 / 1.0000 / 0.2857	0.2105 / 1.0000 / 0.3478	0.9168
3	0.0669	0.1449 / 0.9091 / 0.2500	0.1029 / 0.7000 / 0.1795	0.1449 / 0.9091 / 0.2500	0.8896
4	0.0000	0.2941 / 1.0000 / 0.4545	0.1875 / 0.7500 / 0.3000	0.2941 / 1.0000 / 0.4545	0.9064
5	0.0000	0.0233 / 0.3333 / 0.0435	0.0000 / 0.0000 / 0.0000	0.0233 / 0.3333 / 0.0435	0.7973

Generative Model: gemma-3-1b-it (reference answer 同(4))

RAG: all-MiniLM-L6-v2

RAG pipeline 版本: 改善後 RAG (參考上面 RAG 改善方向)

With RAG — 評比指標總表 (ROUGE 各別欄位)

Test Case	BLEU Score	ROUGE-1 (P / R / F1)	ROUGE-2 (P / R / F1)	ROUGE-L (P / R / F1)	BERTScore F1
1	0.1241	0.1714 / 1.0000 / 0.2927	0.1471 / 1.0000 / 0.2564	0.1714 / 1.0000 / 0.2927	0.9162
2	0.0564	0.1053 / 1.0000 / 0.1905	0.0811 / 1.0000 / 0.1500	0.1053 / 1.0000 / 0.1905	0.9020
3	0.1146	0.2500 / 0.9091 / 0.3922	0.1795 / 0.7000 / 0.2857	0.2500 / 0.9091 / 0.3922	0.9151
4	0.0000	0.2667 / 0.8000 / 0.4000	0.1429 / 0.5000 / 0.2222	0.2667 / 0.8000 / 0.4000	0.8932
5	0.0000	0.0417 / 1.0000 / 0.0800	0.0282 / 1.0000 / 0.0548	0.0417 / 1.0000 / 0.0800	0.8175

結論: 改善後的 RAG pipeline 在文字重疊、語義一致性與檢索精確度上皆優於原始版本，對生成模型 gemma-3-1b-it 提供更有效的知識支援。尤其在 BERTScore 與 ROUGE 之 F1 上的全面提升，表明此 RAG 調整成功強化了語意對齊與內容一致性，為後續 RAG 最佳化奠定穩定基礎。